Assignment 2 Report

AI651 — Deep Learning for Space, Time and Graphs

Roll number: 25280044

Fall 2026

Roll number: 25280044
GitHub (public): `https://github.com/junaid074/DL4STG-PA2`

Question 1 is the river design (no code).
Question 2 lives under `Question2/`. I trained on an Apple M2 Max with PyTorch MPS.
For every learned model I report mean (seed 0, seed 1). Errors to three significant figures.

1 Question 1 — River Forecasting Proposal

1.1 1. Architecture diagram

I sketched the pipeline as a line along the river, with side inputs joining where they physically belong:

```
rain R1,R2 (history) ----\
tributary T1 (history) ---+-- align to 10-min grid --> exogenous encoder
reservoir releases -------/     (past + announced future)
                                          |
levels S1..S4 (history) ---- per-station temporal encode
                                          |
            S1 --> S2 --> S3 --> S4   (message passing along the river)
                  ^ T1 joins before S2
                                          |
                       decode --> 6 steps ahead at S1..S4
```

Rough tensor sizes (batch left out): levels $5 \times L$ (S1–S4 plus T1), rain $2 \times L$, releases $L+H$, station latents $4 \times L \times d$, forecast $4 \times H$. I use $L = 36$ (6 h), $H = 6$ (1 h), d around 64. Rain/releases get projected and added at the stations they affect; releases hit S1 first.

1.2 2. Assumptions

- **A1.** After a bit of interpolation, everything sits on a shared 10-minute clock. That is normal for this kind of monitoring setup; it only affects alignment.
- **A2.** The next hour of announced releases is known when we issue (the brief says so).
- **A3.** Travel time is not a fixed constant — it drifts with how high the river is. I keep a simple prior $\hat{\tau}_{ij} \propto \text{dist}_{ij}$ and let the network correct it.
- **A4.** The unmonitored tributary between S2 and S3 is just a residual inflow. I am not inventing a fake gauge for it.

- **A5.** Six hours of history is enough for recent waves and release effects.
- **A6.** Noise looks roughly additive per station, so Huber loss is fine. A full probabilistic flood model would be nice later, not required for this pitch.

1.3 3. Forecasting task and data specification

Every 10 minutes at issuance time t_0 I look at the last L steps and predict $H = 6$ future levels at S1–S4 only (one hour ahead).

Name	Shape	What / when
levels S1–S4, T1	$B \times 5 \times L$	past, up to t_0
rain R1, R2	$B \times 2 \times L$	past only
releases	$B \times (L + H)$	past + announced future
graph	edges + distances	fixed
output	$B \times 4 \times H$	future S1–S4

At issuance I do *not* get future rain or future levels. I do get the announced releases. If a rain gauge is late I reuse the last reading and flip a missingness bit.

1.4 4. Model computations and design justification

Each station gets a short temporal encoder $z_i = \mathrm{Enc}_T(Y_i, e_i)$. Then a few hops along the river:

$$\tilde{z}_i = z_i + \sum_{j \to i} \alpha_{ji}\, \psi(z_j, z_i, \mathrm{dist}_{ji}, \hat{\tau}_{ji}).$$

With degree at most two this is cheap. I want $K \geq 3$ so S1 can actually reach S4. The head turns $(\tilde{z}_i, U_{\text{future}})$ into six steps. Train with mean Huber over S1–S4 and the horizon.

Who affects whom: releases and upstream rain show up at S1, then hop down. T1 feeds the edge into S2. S3 never sees the missing tributary directly — only whatever S2 carries plus a residual the net can learn (A4). The travel-time prior is a hint, not a hard delay line.

1.5 5. Operation and exceptional conditions

- **Missing readings.** Keep a mask, fill with the last value. If a station has been dark for a few steps, stop summing it into neighbours but still try to decode it from them.
- **Delayed readings.** Do not wait. Issue on whatever is complete; late packets land in the next cycle.
- **Announced future releases.** Pass them straight into the decoder as known inputs.
- **Schedule vs actual.** When the real release differs, S1 notices on the next step and the error walks downstream. A tiny residual corrector on S1 is optional.

Four stations and $L = 36$ is a small model — easy to run every ten minutes on a normal CPU.

1.6 6. Uncertainty, limitations, and additional information

The unmonitored tributary can still surprise S3/S4 after a local storm; a cheap stage gauge there would help. Travel times change in flood, so the prior goes stale — occasional discharge checks would let me retune $\hat{\tau}$. Rain away from R1/R2 is invisible; radar QPE over the basin would help. Without those extras the system still runs. It just leans harder on recent downstream levels and will be weaker on rare tributary-driven events.

2 Question 2 — Motion or Memory?

Table 1 — transform and window checks

Check	Expected	Yours
(a) $\max \lvert M(M(x)) - x \rvert$	0	0
(b) $\max \lvert R^4(x) - x \rvert$	0	0
(c) $\max \lvert MRM(x) - R^{-1}(x) \rvert$	0	0
(d) bright pixel of $M(p)$	(7, 10, 43)	(7, 10, 43)
(e) bright pixel of $R(p)$	(7, 43, 10)	(7, 43, 10)
(f) 1× input / target	1–5 / 6–10	1–5 / 6–10
(g) 2×	1,3,5,7,9 / 11,13,15,17,19	same
(h) 3×	1,4,7,10,13 / 16,19	same

All eight matched before I trained anything.

Figure 1, Q2.1, Q2.2

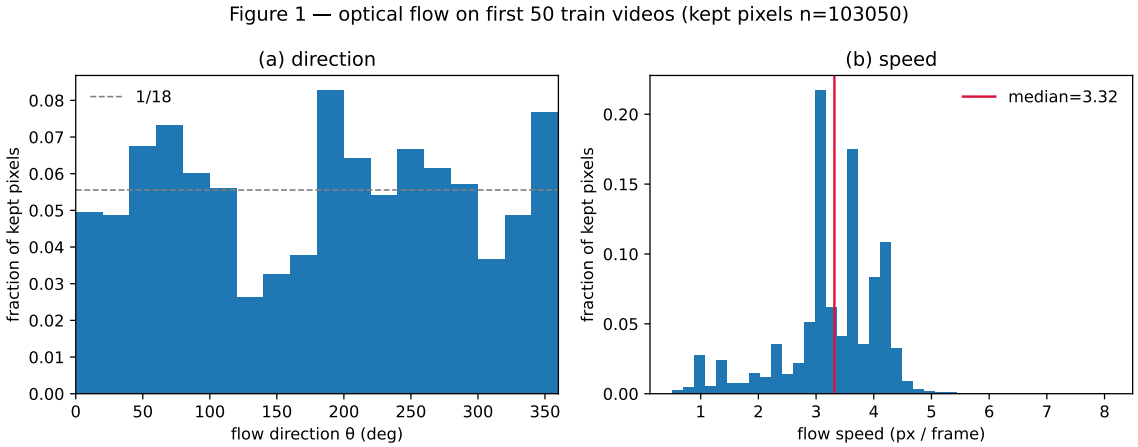


Figure 1: Figure 1 — Farneback flow on the first 50 training videos. Kept $n = 103050$ pixels; median speed 3.32 px/frame.

Q2.1. The direction histogram is not perfectly flat at 1/18, but there is mass in a lot of bins. So if I mirror or rotate the training set, I mostly reshuffle directions that already showed up. Typical speed is about 3.3 px/frame, which means 2× ≈ 6.6 and 3× ≈ 9.9. Overlaps make the flow noisy, but with this many kept pixels I would not change the answer.

Q2.2. Mirror and rotation leave the motion rule alone (straight lines, wall bounces) but change how the digits look. Speed-up keeps the look and changes how far a digit jumps between frames. That is why the three tests pull apart appearance, unfamiliar speed, and reach.

Model A, reach, Table 2, Figure 2

Q2.3. After `pixel_unshuffle(4)` one grid cell is 4 original pixels. A 3×3 moves information by one cell. In one time step the longest path is input → layer 1 → layer 2 → head, so two cells = 8 pixels. I take reach$(A) = 8$ px per step.

Model	MSE_1	MSE_2	MSE_3	MSE_4	
A	0.0169 (0.0173, 0.0166)	0.0202 (0.0207, 0.0198)	0.0224 (0.0229, 0.0219)	0.0241 (0.0247, 0.0236)	0.02
Blank	0.0426	0.0426	0.0426	0.0425	
Copy-last	0.0492	0.0604	0.0669	0.0708	

Table 2 (test). Model A has 480,784 trainable parameters.

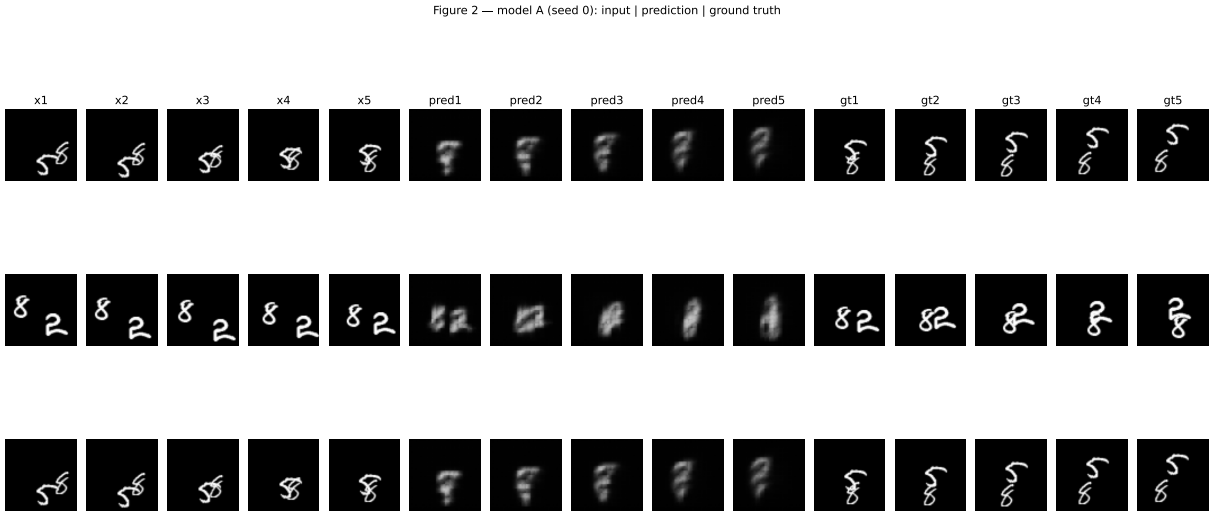


Figure 2: Figure 2 — model A, seed 0. Inputs, prediction, and ground truth on three test videos (one with overlapping digits).

Q2.4. A is clearly better than Blank and Copy-last on every horizon. Error creeps up with h (about 0.017 to 0.026). When digits overlap the forecast goes soft — two blobs melt into one blur instead of staying as two digits.

Q2.5 — written before the mirror / rotation / speed tests

(Unedited. Committed in the repo before those runs.)

I am writing this before any mirror, rotation, or speed-up test.

Mirror: I do not think model A will pass. The motion rule is the same after a flip, but the digits look different, and training never asked the net to treat a mirrored 2 like a 2. Expect a non-zero C_M and a clear rise in E_M.

Rotation: same story, probably worse. Expect C_R, E_R above the mirror numbers.

Speed: reach ≈ 8 px/step. If typical motion is only a couple of pixels per frame at 1×, then 2× may still be close and 3× is where it should break (blur, stall, or a digit left behind).

Tables 3–4, Figures 3–5

Model	$E(5)$	C_M	E_M	C_R
A	0.0219 (0.0224, 0.0214)	0.00176 (0.00175, 0.00177)	0.0225 (0.0230, 0.0221)	0.00315 (0.00307, 0.00323)
Blank	0.0426	0	0.0426	0
Copy-last	0.0641	0	0.0641	0

Table 3 — $1\times$, $t = 1$.

Figure 3 — largest C_M videos (model A seed 0)

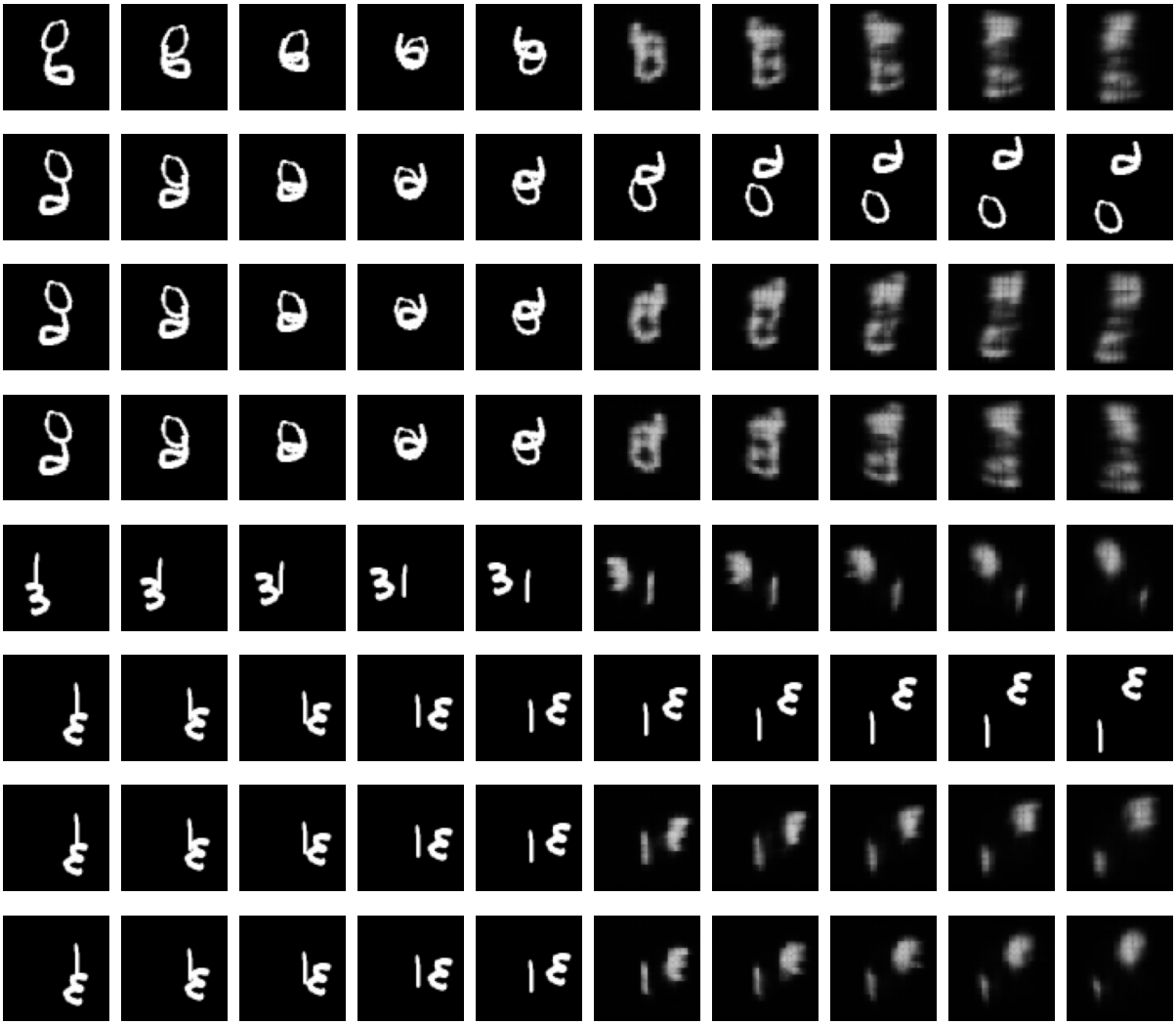


Figure 3 — largest C_R videos (model A seed 0)



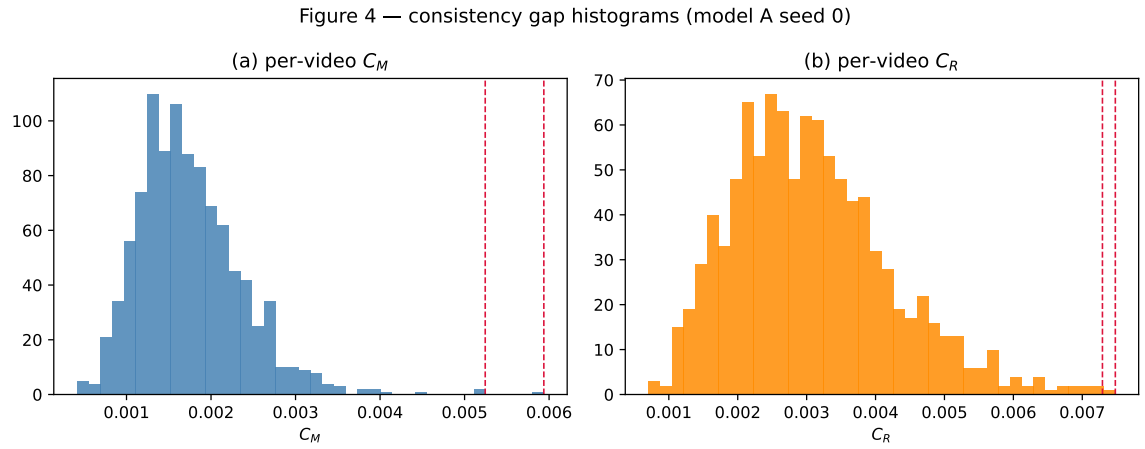


Figure 4: Figure 4 — histograms of per-video C_M and C_R for model A seed 0. The Figure 3 videos are marked.

Q2.6. Blank and Copy-last both get $C_T = 0$. That is not surprising: all zeros stay zeros, and copying the last frame then flipping is the same as flipping then copying. So a zero consistency gap does not mean the predictor is good — you still need E_T next to it.

Q2.7. Looking at one of the high-C_M panels in Figure 3, rows 3 and 4 disagree. $f(T(X))$ sends a digit the wrong way after the wall, while $T(f(X))$ keeps the flipped bounce. So the network's idea of motion is still tied to how the digit is drawn, not only to velocity.

Q2.8. Rotation is a bit harder (C_R and E_R sit above the mirror numbers). From Q2.1 and Q2.2 I would put most of that on appearance: directions were already somewhat familiar after a 90° turn, but the glyphs change more under rotation than under a mirror.

Model	$E(2)$ 1×	$E(2)$ 2×	$E(2)$ 3×	$E(5)$ 1×	
A	0.0186 (0.0190, 0.0182)	0.0441 (0.0435, 0.0448)	0.0511 (0.0505, 0.0516)	0.0219 (0.0224, 0.0214)	0.04
Copy-last	0.0548	0.0661	0.0703	0.0641	

Table 4.

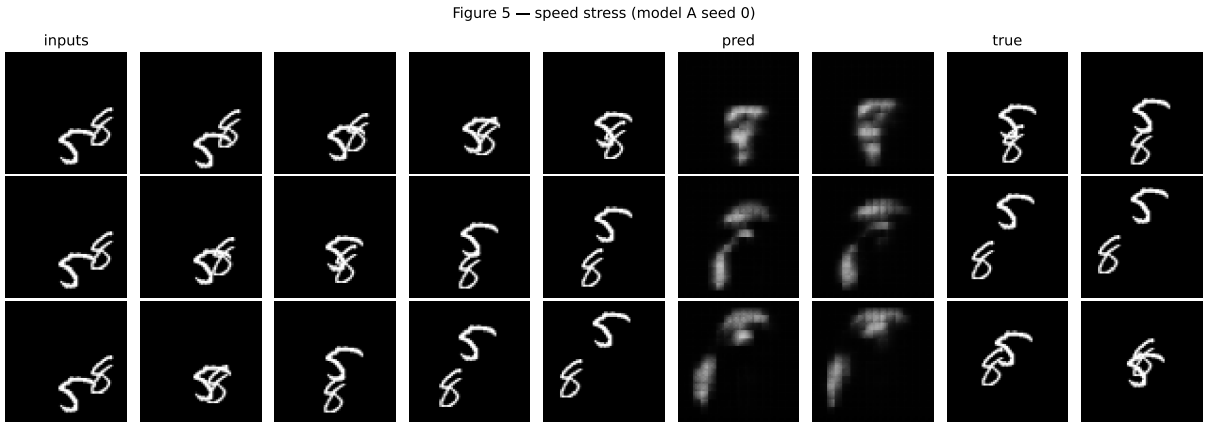


Figure 5: Figure 5 — test video 0 at 1×, 2×, 3× (model A seed 0). Five inputs, first two predictions, first two true frames.

Q2.9. At 1× A is fine. At 2× it gets much worse, and at 3× $E(2) \approx 0.051$ — basically falling apart. Median speed 3.3 with reach 8 means 3× ≈ 9.9 is past what one step can connect, so that matches Q2.3. In Figure 5 at 3× the digits blur and stall instead of jumping the full gap.

Q2.10. From A alone I cannot tell if the speed failure is "never saw fast motion in training" or "the net cannot reach that far". I would train the same architecture with speed augmentation (C) and a larger-reach net also with speed aug (D). If C is still bad at 3× and D is better, the leftover problem is reach.

Q2.11 — written before training B–D

(Unedited. Same early commit.)

B (symmetry aug) should mainly fix mirror/rotation (appearance). C (speed aug) should help 2×/3× only up to what reach allows. D (larger reach + speed aug) is the one that should handle 3× if the speed failure was architectural.

Models B–D, Table 5

Model D keeps the same parameter count as A (480,784) but uses dilation 2 on the 3×3 ConvLSTM kernels. That gives radius 2 cells per layer, so two layers from the input reach 16 px/step (A was 8), still under the 721,176 cap.

	$E(5)$ 1×	C_M	E_M	C_R	E_R
A	0.0219 (0.0224, 0.0214)	0.00176 (0.00175, 0.00177)	0.0225 (0.0230, 0.0221)	0.00315 (0.00307, 0.00323)	0.0237 (0.0241, 0.0233)
B	0.0221 (0.0223, 0.0220)	0.00119 (0.00134, 0.00104)	0.0222 (0.0223, 0.0220)	0.00160 (0.00197, 0.00123)	0.0222 (0.0223, 0.0220)
C	0.0268 (0.0265, 0.0271)	0.00122 (0.00121, 0.00123)	0.0271 (0.0268, 0.0274)	0.00244 (0.00191, 0.00297)	0.0280 (0.0275, 0.0284)
D	0.0283 (0.0281, 0.0285)	0.00133 (0.00133, 0.00133)	0.0285 (0.0283, 0.0287)	0.00221 (0.00238, 0.00204)	0.0294 (0.0293, 0.0295)

Table 5. Params are 480,784 for A–D. Mean train time per epoch (s): 10.7, 10.6, 10.5, 13.1. Mean inference time on the 1000 test videos, batch 32 (s): 0.73, 0.73, 0.73, 0.89.

Q2.12. See above — dilation 2, reach 16 vs 8 for A.

Q2.13. Mirror / rotation look like an *appearance* issue: B lowers C_M and C_R without really helping speed. Fast motion is partly *unfamiliar data*: C cuts $E(2)$ at 3× from about 0.051 down to 0.034. What is left looks like *reach*: D gets a little further (0.031). Extra 3× windows cannot move information farther than one step's receptive field, so C plateaus above D.

Q2.14. Speed aug does cost accuracy on plain 1× clips (C/D around 0.027 vs A's 0.022). I think that trade is fine if you care about faster motion; if you only cared about 1×, A or B would be the better pick.

Final model (symmetrised)

For any trained f,

$$\bar{f}(X) = \frac{1}{8} \sum_{g \in G} g^{-1}\big(f(g(X))\big).$$

Because G is a finite group, reindexing the sum shows $\bar{f}(M(X)) = M(\bar{f}(X))$ and the same for R, so $C_M = C_R = 0$ exactly (up to float noise). I wrap **model D** (handout default). Table 5 already gave D a small 3× edge over C, and I want the exact symmetry.

Model	$E(5)$ 1×	C_M	E_M	C_R	E_R	$E(2)$ 3×
sym B	0.0215 (0.0215, 0.0215)	4.09×10^{-17}	0.0215	1.61×10^{-17}	0.0215	0.0475 (0.0454, 0.0495)
sym D	0.0281 (0.0279, 0.0283)	3.39×10^{-17}	0.0281	1.34×10^{-17}	0.0281	0.0309 (0.0308, 0.0310)

Table 6. C_T reported in scientific notation. Inference ≈ 3.4 s for sym B and ≈ 4.6 s for sym D. Training time is just the time of the underlying B or D — wrapping does not retrain.

Figure 3 — largest C_M videos (model A seed 0)

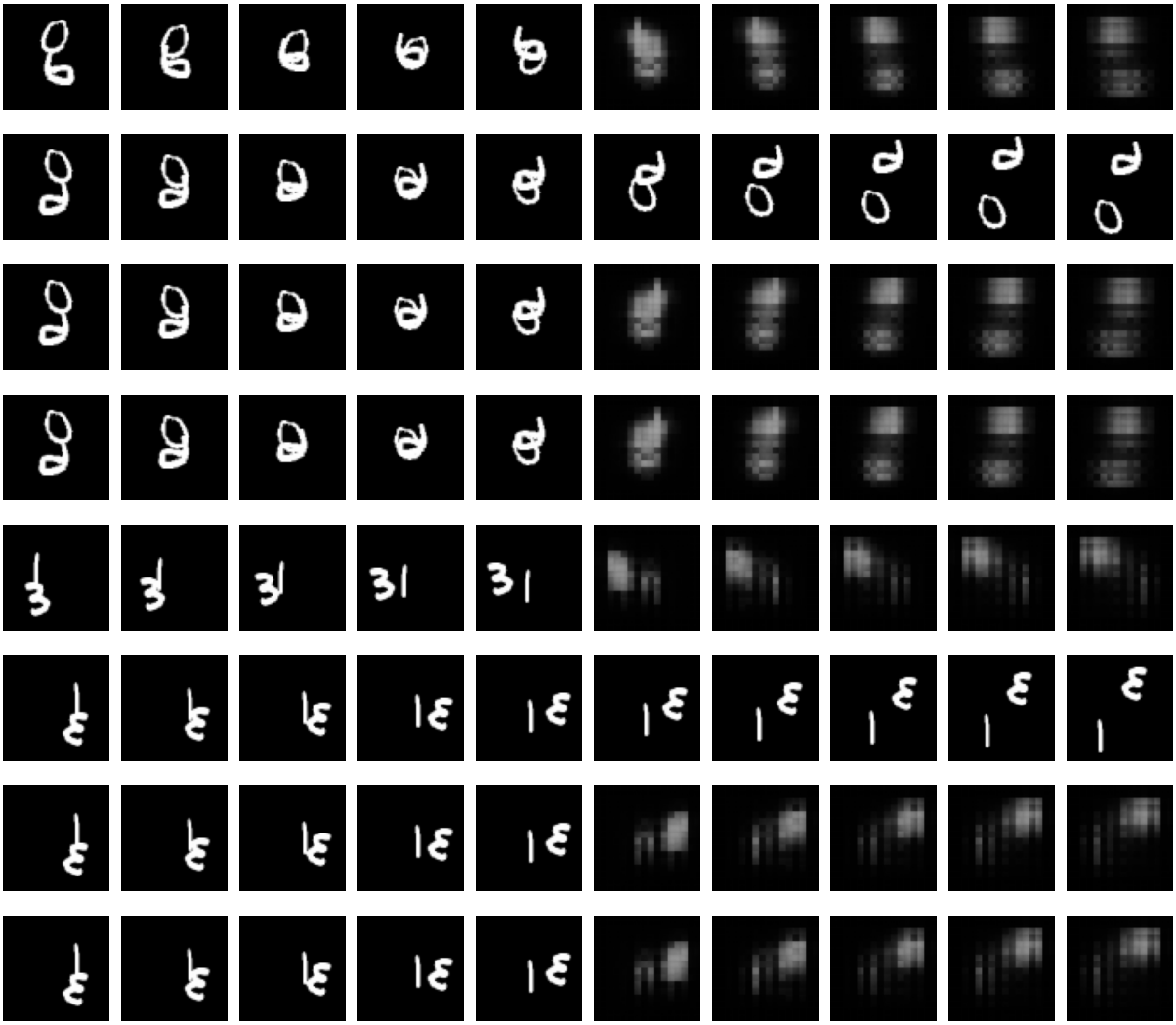
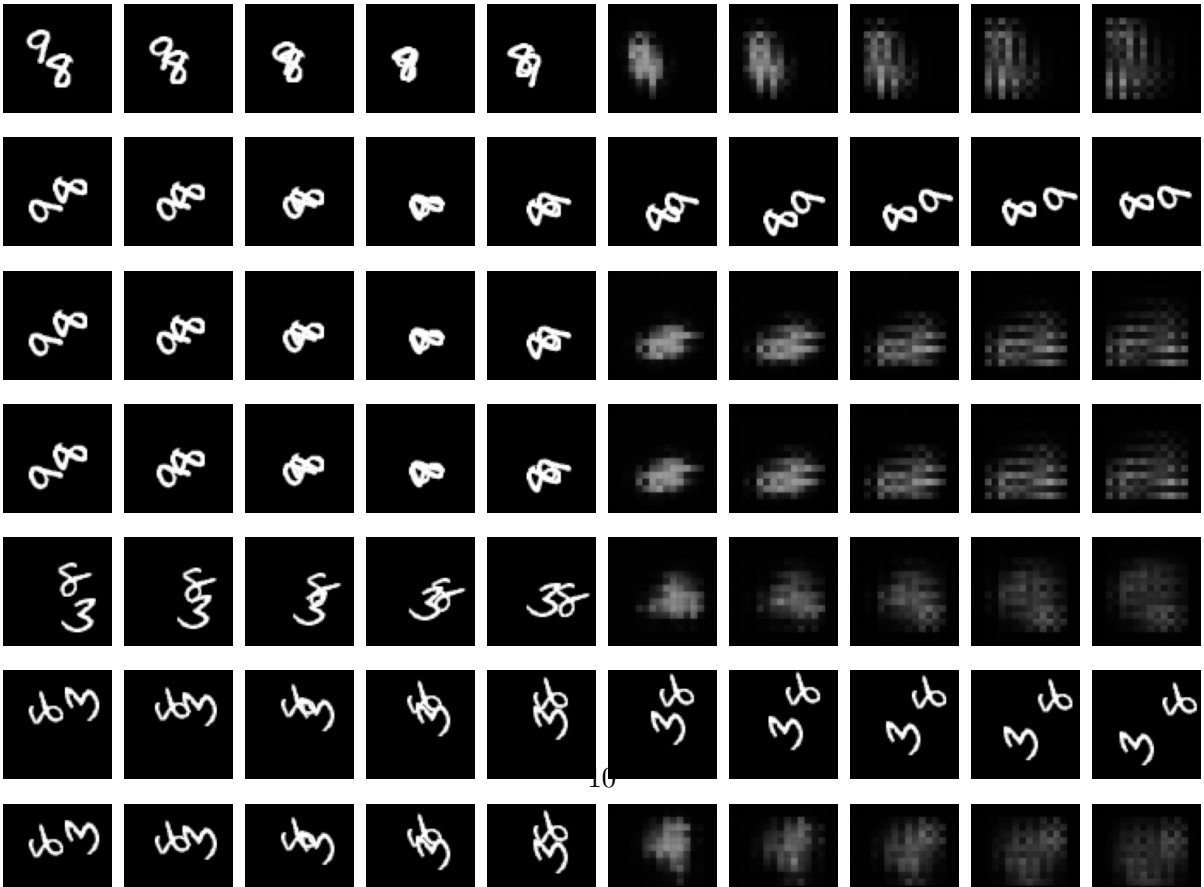


Figure 3 — largest C_R videos (model A seed 0)



Q2.15. Symmetry from data (B) helps but does not drive C_T to zero. Building it in with $\bar{f}$ does ($C_T \sim 10^{-17}$), at roughly 8× the forward cost. Averaging eight views can blur when they disagree; on these runs sym B did not get worse at 1×, and sym D kept the better 3× number.

Q2.16. On the Q2.7 video, rows 3 and 4 now match — they have to, by construction. The wrong-way bounce from appearance is gone. What is left at high speed is blur, and Figure 6b looks cleaner at 3× than Figure 5 did.

Q2.17. (under 150 words) Symmetrised D is the closest thing I have to "learned motion" in this homework: exact mirror/rotation consistency, and much better 3× tracking than A. I would still not call it done — a skeptic could ask for 4× or a path that is not a straight bounce. One clear gap: it does not match A's plain 1× error while staying good at 3×. The speed-aug trade-off is still sitting there.

AI tools

I brainstormed both questions myself first (Q1 headings and assumptions; for Q2 the Table 1 plan, the reach count, and the Q2.5 / Q2.11 guesses — those two were committed before the matching experiments). After that I asked a chat assistant to help wire the solution in small steps (loaders, train loop, eval / plots), not as one big dump.

Prompt (rough): I already have a plan for transforms and model A; help me write the next piece (e.g. the training loop, then the C_T/E_T metrics).
Got: a sketch for that piece.
What I kept / changed: Table 1 checks, reach write-up, Model D as dilated ConvLSTM, the pre-experiment Q2.5/Q2.11 text, and the Q1 design.